01 Supervised Learning Basics

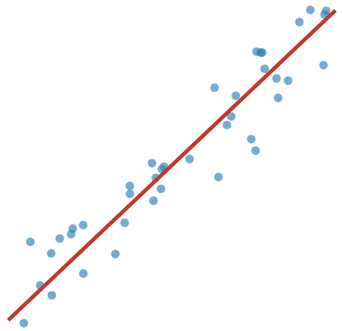
Core algorithms that learn a mapping from labelled inputs to outputs

github.com/ranjiGT/ai-machine-learning-codes

01 Linear Regression

$\hat{y} = \mathbf{w}^T\mathbf{x} + b$

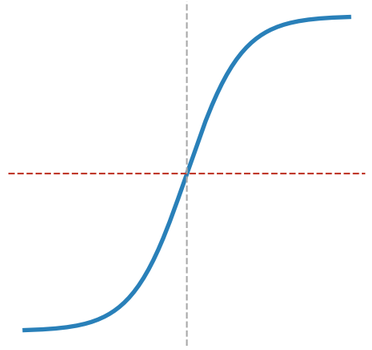
- Model: $\hat{y} = w^T x + b$
- Loss: MSE = $\Sigma(y-\hat{y})^2/n$
- Normal eq: $w=(X^TX)^{-1}X^Ty$



02 Logistic Regression

$P(y=1) = \sigma(\mathbf{w}^T\mathbf{x})$

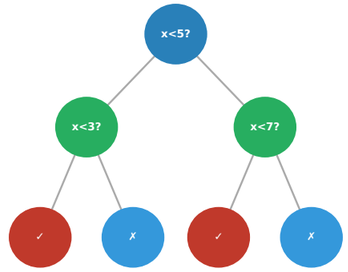
- $\sigma(z)=1/(1+e^{-z})$ squashes to [0,1]
- Loss: binary cross-entropy
- Decision boundary: $w^Tx+b=0$



03 Decision Trees

$IG = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$

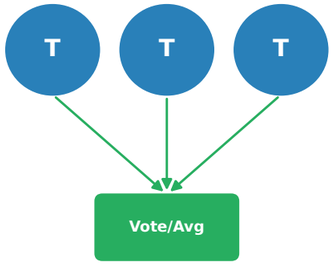
- Splits on best impurity gain
- Greedy, top-down induction
- Prone to overfitting



04 Random Forest

$\hat{f} = \frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x})$

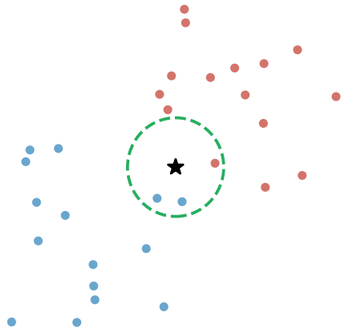
- Bagging of decision trees
- Random feature subset per split
- Reduces variance via averaging



05 K-Nearest Neighbours

$d(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|_2$

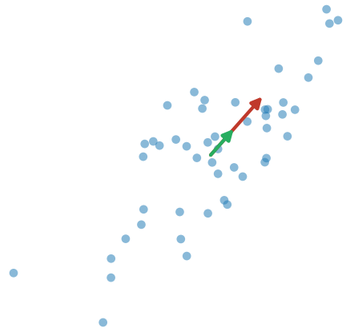
- Label = majority vote of k neighbours
- No training — lazy learner
- Sensitive to scale & k



06 Principal Component Analysis

$\mathbf{z} = \mathbf{W}^T(\mathbf{x} - \mu)$

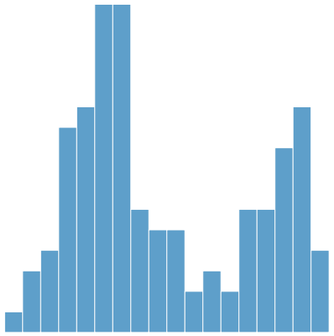
- Projects to axes of max variance
- Eigenvectors of cov matrix
- Reduces dimensionality



07 Naïve Bayes

$\hat{y} = \arg\max_y P(y)\prod_i P(x_i|y)$

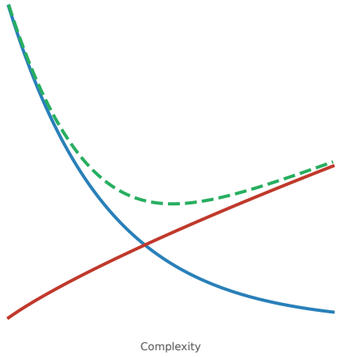
- Assumes feature independence
- $P(y|x) \propto P(y) \cdot \Pi P(x_i|y)$
- Fast & effective on text



08 Underfitting vs Overfitting

$E_{total} = \text{Bias}^2 + \text{Var} + \sigma^2$

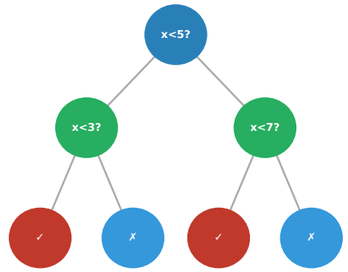
- Underfitting: high bias, low variance
- Overfitting: low bias, high variance
- Sweet spot minimises total error



09 Reduced Error Pruning

Prune if $Acc_{pruned} \geq Acc_{full}$

- Remove subtrees that don't hurt accuracy
- Uses a held-out pruning set
- Greedy bottom-up simplification



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

02 Evaluation & Model Selection

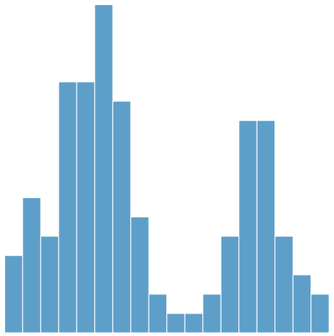
Techniques for estimating generalisation performance and selecting models

github.com/ranjiGT/ai-machine-learning-codes

10 Random Sampling

$Acc = \frac{TP + TN}{N}$

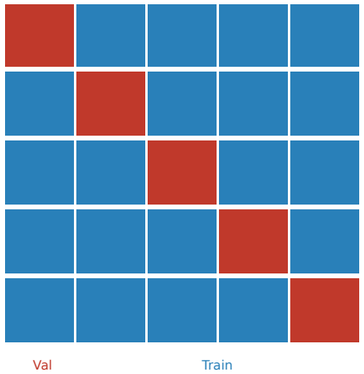
- Random train/test split
- Quick but high variance
- Stratify to preserve class ratios



11 Cross-Validation

$\widehat{Err} = \frac{1}{k}\sum_{i}^{k} Err_i$

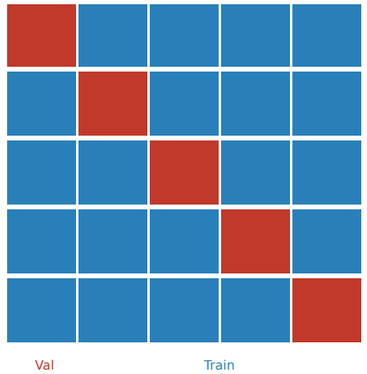
- Split data into k folds
- Train on k−1, test on 1
- Average over all k rotations



12 Nested Cross-Validation

$\hat{E} = \frac{1}{K_o}\sum E_{\text{inner-best}}$

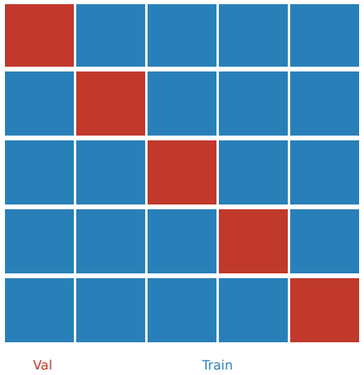
- Outer loop: estimate performance
- Inner loop: hyperparameter tuning
- Unbiased estimate of tuned model



13 Hold-Out Method

$\text{Error} = \frac{1}{|D_{test}|}\sum \mathcal{L}(y, \hat{y})$

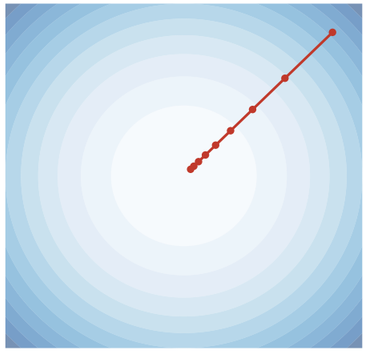
- Typical split: 70/30 or 80/20
- Fast but wastes data
- Use only for large datasets



14 Hyperparameter Tuning

$\theta^* = \arg\min_{\theta} E_{val}(\theta)$

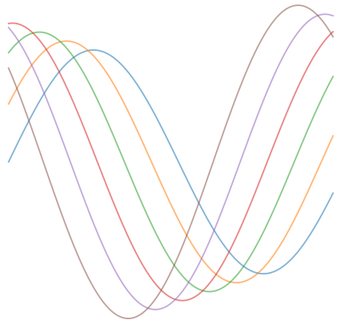
- Grid search / random search
- Bayesian optimisation
- Always tune on validation set



15 Bootstrap Evaluation

$\hat{E}_{.632} = 0.368 \cdot \hat{E}_{train} + 0.632 \cdot \hat{E}_{oob}$

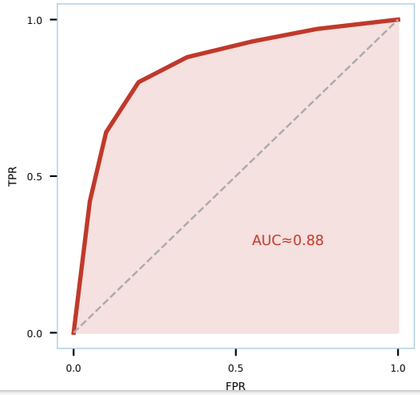
- Sample n points with replacement
- Estimate confidence intervals
- 0.632 bootstrap corrects optimism bias



108 Regression Evaluation Metrics

$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$

- MAE: mean absolute error
- RMSE: penalises large errors
- R^2: fraction of variance explained



03 Tree-Based & Rule-Based Methods

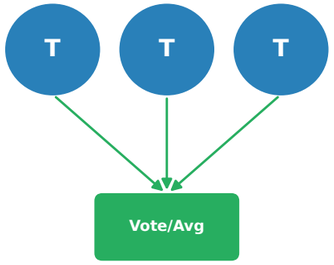
Algorithms that build interpretable rule sets from training data

github.com/ranjiGT/ai-machine-learning-codes

16 Ensemble Methods

$H(\mathbf{x}) = \text{sign}\left(\sum_m \alpha_m h_m(\mathbf{x})\right)$

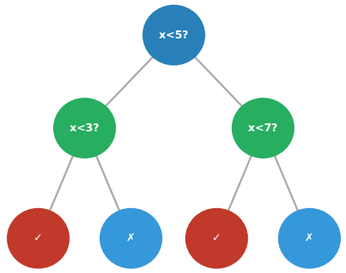
- Bagging reduces variance (RF)
- Boosting reduces bias (AdaBoost)
- Stacking combines diverse models



17 Hunt's Algorithm

$\text{Split}^* = \arg\max_S \Delta\text{Impurity}$

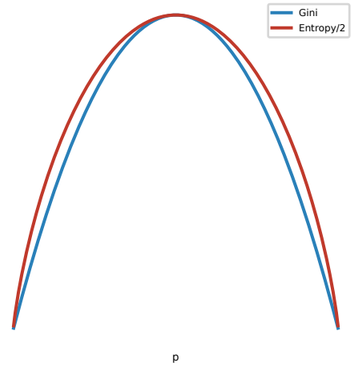
- Basis of most tree learners
- Recursively split training set
- Stop when node is pure / small



18 Impurity Measures

$\text{Gini} = 1 - \sum_k p_k^2$

- Gini: $1-\Sigma p_k^2$
- Entropy: $-\Sigma p_k \log p_k$
- Misclassification: $1-\max p_k$



19 Rule-Based Classifier

$R : (A_1 \wedge A_2) \Rightarrow y$

- IF-THEN rules, easy to interpret
- Coverage: fraction rules apply to
- Accuracy: fraction correctly classified

20 Sequential Covering

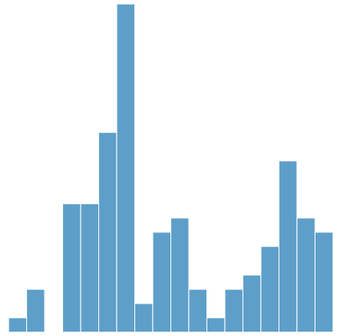
$\text{FOIL gain} = t\left(\log \frac{p_1}{p_1+n_1} - \log \frac{p_0}{p_0+n_0}\right)$

- Learn one rule, remove covered
- Repeat until set is covered
- Order-sensitive rule list

65 Laplace Correction

$P(x_i|y) = \frac{n_{iy}+1}{n_y+|V|}$

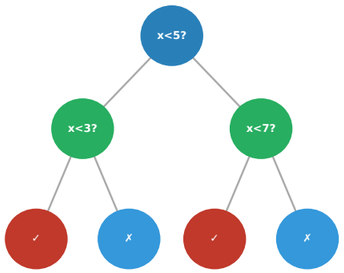
- Avoids zero probabilities
- Add 1 to each count (add-1 smoothing)
- Posterior: $(n_k+1)/(n+|V|)$



79 ID3 Algorithm

$IG(S,A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$

- Splits on maximum Information Gain
- Uses entropy as impurity measure
- Only handles discrete features



04 Probabilistic & Bayesian Learning

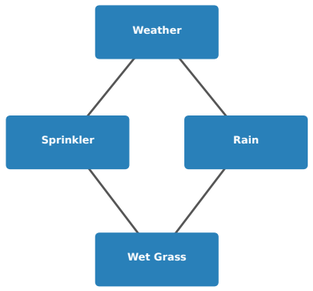
Principled probabilistic inference and uncertainty-aware models

github.com/ranjiGT/ai-machine-learning-codes

21 Bayesian Belief Networks

$P(\mathbf{x}) = \prod_i P(x_i \mid pa(x_i))$

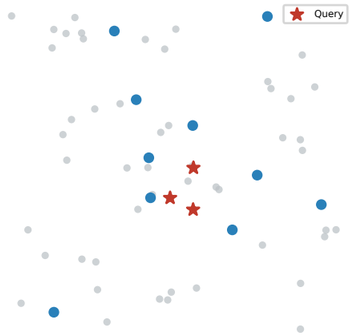
- DAG encoding conditional independences
- Joint: P(x)=ΠP(x_i|parents(x_i))
- Inference by variable elimination



51 Uncertainty Sampling

$x^* = \arg\min_x P(\hat{y}|x)$

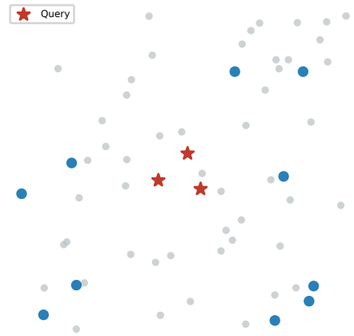
- Query the sample model is least sure about
- Margin: P(y_1|x)−P(y_2|x) small
- Entropy: −Σ P(y|x)log P(y|x) large



52 Query by Committee

$x^* = \arg\max_x H_{\text{vote}}(x)$

- Maintain committee of hypotheses
- Query where committee disagrees most
- Vote entropy: −Σ(V/C)log(V/C)



53 Expected Model Change

$x^* = \arg\max_x \mathbb{E}[\|\nabla L\|]$

- Query that causes biggest gradient
- Maximise ||∇L(θ)||
- Computationally expensive

54 Expected Error Reduction

$x^* = \arg\min_x \mathbb{E}[E_{gen}]$

- Query that most reduces future error
- Average over possible labels
- Expensive: retrains for each candidate

55 Variance Reduction

$x^* = \arg\min_x \mathrm{Var}[\hat{y}]$

- Focus on output-space variance
- Fisher information criterion
- Related to experimental design

56 Information Density

$x^* \propto \phi_U(x) \cdot \left(\frac{1}{D}\sum_u \mathrm{sim}(x, x^u)\right)^\beta$

- Weight uncertainty by representativeness
- Dense regions get more queries
- ID: US(x)·(avg sim)^β

05

Neural Networks Fundamentals

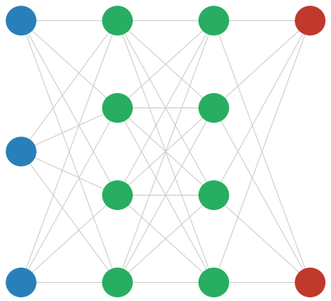
Biologically inspired models built from layers of interconnected nodes

github.com/ranjiGT/ai-machine-learning-codes

22 Artificial Neural Networks

$\mathbf{h} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b})$

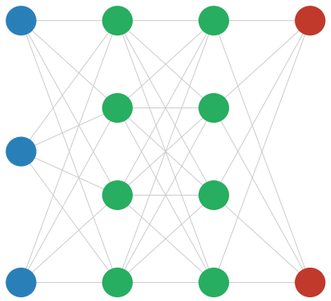
- Layers of weighted sums + activations
- Universal approximator (1 hidden layer)
- Trained by gradient-based optimisation



23 Feed Forward NNs

$\mathbf{y} = f^{(L)} \circ \cdots \circ f^{(1)}(\mathbf{x})$

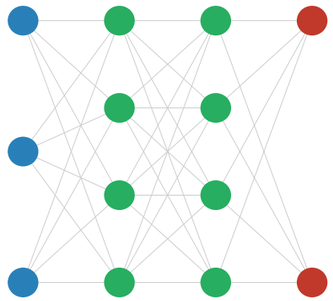
- Information flows input→output only
- No cycles or recurrent connections
- Multi-layer perceptron (MLP)



91 Intro to Deep NNs

$f(\mathbf{x};\theta) = f_L(\ldots f_1(\mathbf{x}))$

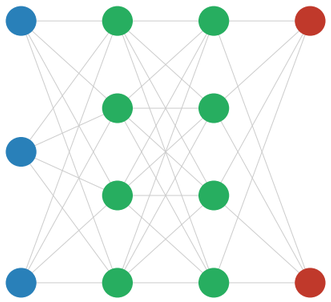
- Many hidden layers → deep learning
- Hierarchical feature learning
- Needs large data & GPUs



92 Structure of a Neuron

$a = \phi\left(\sum_i w_i x_i + b\right)$

- Receives weighted inputs: z=Σwᵢxᵢ+b
- Non-linear activation: a=φ(z)
- Biological analogy: dendrites→axon



93 Weight Matrix in NNs

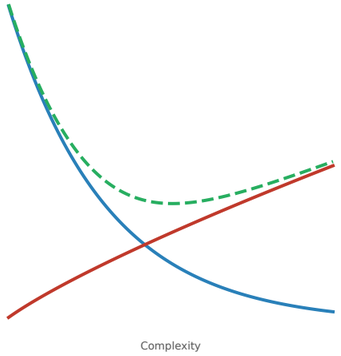
$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$

- W[l] maps layer l−1 to l
- Dims: (n_l, n_{l−1})
- Shared weights → parameter efficiency

94 Generalisation of NNs

$\mathcal{L}_{reg} = \mathcal{L} + \lambda\|\mathbf{W}\|_F^2$

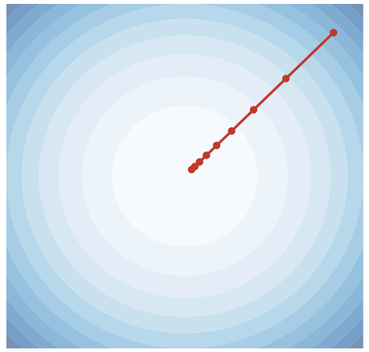
- Dropout, weight decay, early stopping
- Implicit regularisation by SGD noise
- Bias–variance trade-off applies



95 Learning Neural Networks

$\mathbf{W} \leftarrow \mathbf{W} - \eta\nabla_{\mathbf{W}}\mathcal{L}$

- Backprop + SGD / Adam
- Mini-batch gradient descent
- Learning rate scheduling



96 Vectorization of NNs

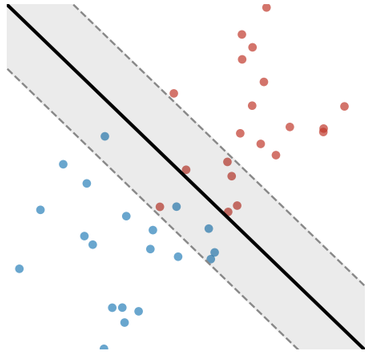
$\mathbf{Z} = \mathbf{W}\mathbf{X} + \mathbf{b}\mathbf{1}^T$

- Batch all examples: Z=WX+b
- GPU parallelism over batch dim
- Eliminates explicit Python loops

97 Decision Boundary

$\{x : f(\mathbf{x}) = 0.5\}$

- Surface where P(y=1|x)=0.5
- Highly non-linear for deep nets
- Controlled by weights & activations



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

06 Training Deep Networks

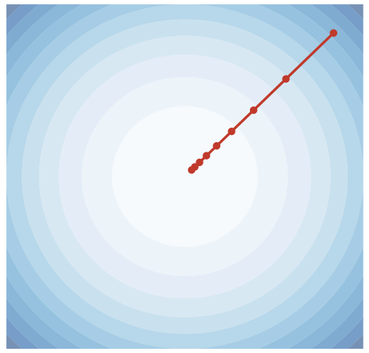
Optimisation algorithms and practical techniques for training deep models

github.com/ranjiGT/ai-machine-learning-codes

80 Gradient Descent

$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}$

- Move in direction of negative gradient
- Batch / SGD / mini-batch variants
- Convergence rate depends on curvature



98 The Chain Rule

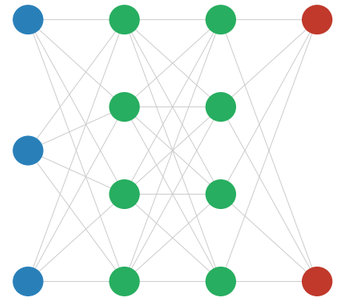
$\frac{dL}{dx} = \frac{dL}{dy} \cdot \frac{dy}{dx}$

- dL/dW = (dL/da)·(da/dz)·(dz/dW)
- Enables computing all gradients
- Foundation of automatic differentiation

99 Backpropagation

$\delta^{[l]} = (\mathbf{W}^{[l+1]T}\delta^{[l+1]}) \odot \phi'(\mathbf{z}^{[l]})$

- Forward pass: compute activations
- Backward pass: propagate δ errors
- O(n) per layer — efficient



100 The Delta Rule

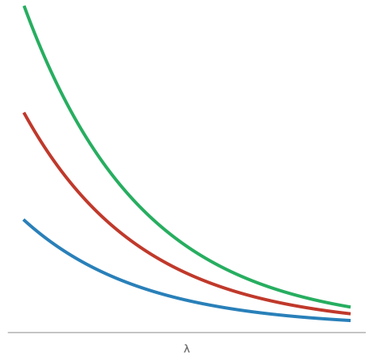
$\Delta w_i = \eta(y - \hat{y})x_i$

- Single-layer online learning rule
- Δw = η(y−ŷ)x — Widrow–Hoff
- Precursor to backpropagation

101 Vanishing/Exploding Gradients

$\|\nabla_1\| \approx \prod_l \|\mathbf{W}^{[l]}\| \cdot \phi'$

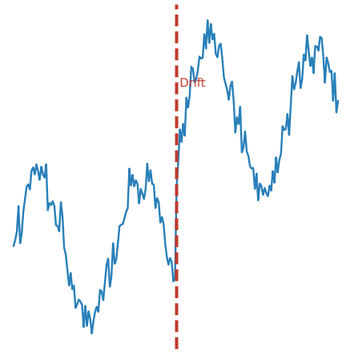
- Gradients shrink/grow through layers
- Sigmoid/tanh worsen vanishing
- Fix: ReLU, BatchNorm, residuals



106 Teacher Forcing

$h_t = f(h_{t-1}, y^*_{t-1})$

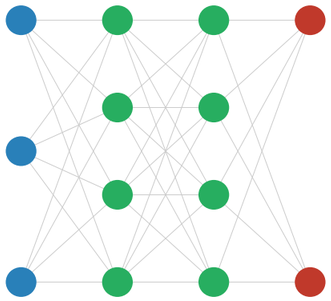
- Feed ground-truth as next RNN input
- Speeds up training convergence
- Can cause exposure bias



107 LSTM

$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$

- Forget, input, output gates
- Cell state acts as long-term memory
- Solves vanishing gradient for sequences



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

07

Activation Functions

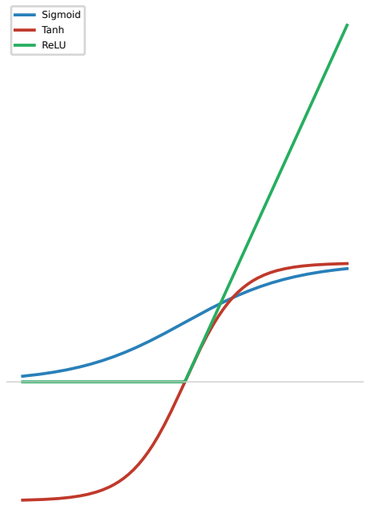
Non-linearities that give neural networks their expressive power

github.com/ranjiGT/ai-machine-learning-codes

102 Step Activation

$\phi(z) = \mathbf{1}[z \geq 0]$

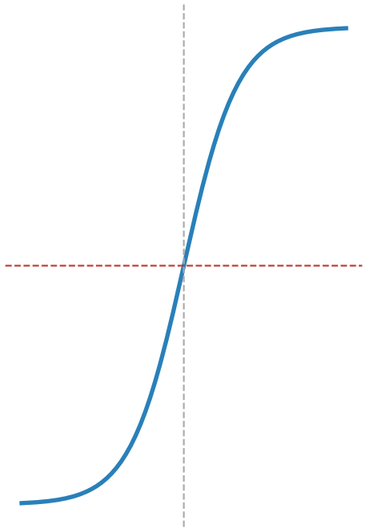
- Binary output: 0 or 1
- Not differentiable at 0
- Used in original perceptron



103 Sigmoid Activation

$\sigma(z) = \frac{1}{1+e^{-z}}$

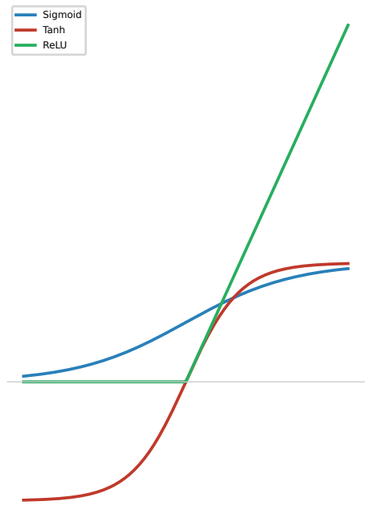
- Smooth, squashes to (0,1)
- Saturates → vanishing gradient
- Used in binary output layers



104 Hyperbolic (Tanh)

$\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$

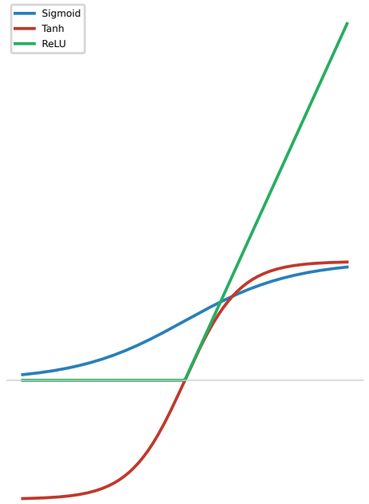
- Zero-centred output in $(-1,1)$
- Better than sigmoid for hidden layers
- Still saturates at extremes



105 ReLU Activation

$\mathrm{ReLU}(z) = \max(0, z)$

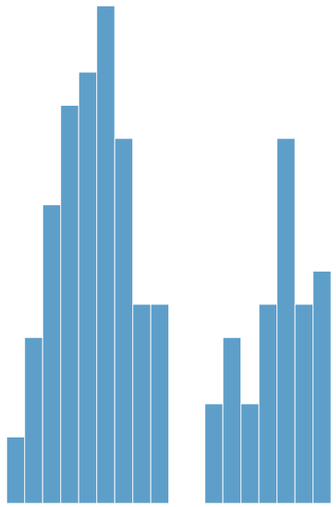
- max(0,z) — cheap to compute
- No saturation for z>0
- Dead ReLU: fix with Leaky ReLU



114 Softmax Activation

$\mathrm{softmax}(z_k) = \frac{e^{z_k}}{\sum e^{z_j}}$

- Converts logits to probabilities
- Outputs sum to 1
- Used in multi-class output layer



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

08 Support Vector Machines

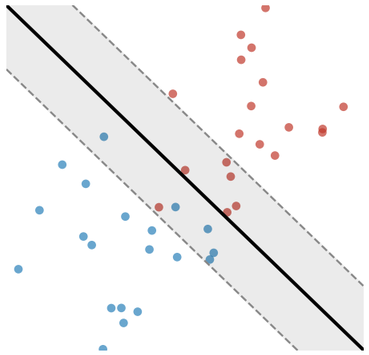
Maximum-margin classifiers with kernel extensions for non-linear data

github.com/ranjiGT/ai-machine-learning-codes

24 Support Vector Machine

$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum\xi_i$

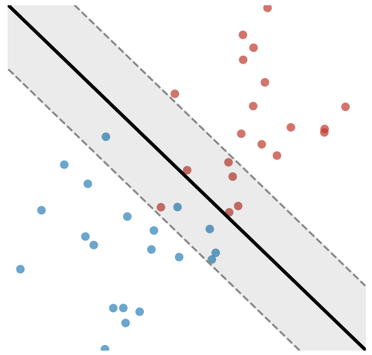
- Maximise margin between classes
- C balances margin vs misclassification
- Only support vectors matter



25 Maximal Margin Classifier

$\text{margin} = \frac{2}{\|\mathbf{w}\|}$

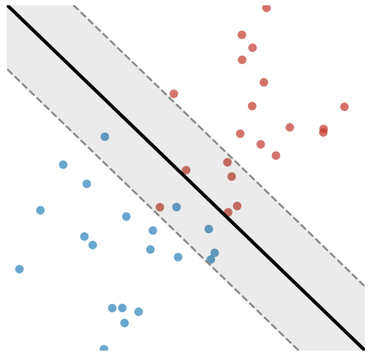
- Hard-margin SVM (linearly separable)
- Unique maximum-margin hyperplane
- Geometric margin = 2/||w||



26 Non-Linear SVM

$K(\mathbf{x},\mathbf{x}') = \exp(-\gamma\|\mathbf{x}-\mathbf{x}'\|^2)$

- Map x→φ(x) to higher dimension
- Kernel trick: K(x,x')=φ(x)·φ(x')
- RBF, polynomial, sigmoid kernels



110 Kernel Function & Matrix

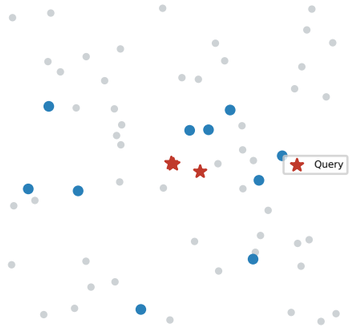
$K_{ij} = \phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j)$

- K must be PSD (Mercer's theorem)
- Gram matrix $K_{ij} = K(x_i,x_j)$
- Enables SVM in infinite-dim spaces

111 Semi-Supervised Learning

$\min_f \frac{1}{T}\sum\mathcal{L} + \lambda R(f,\mathcal{U})$

- Uses few labelled + many unlabelled
- Self-training, co-training, label prop
- Low-density assumption for decision boundary



112 Safe Semi-Supervised

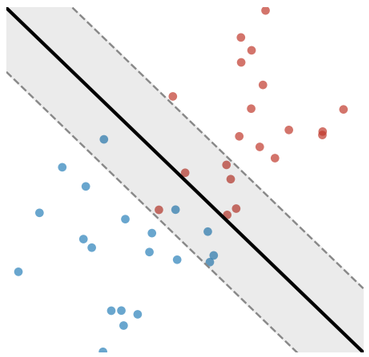
$R_{SSL} \le R_{SL} + \varepsilon$

- SSL should not hurt vs supervised
- Safety constraints on unlabelled usage
- TSVM with safe wrapper

113 S3VM & TSVM

$\min \frac{1}{2}\|\mathbf{w}\|^2 + C\sum\xi + C^*\sum\xi^*$

- Place decision boundary in low-density region
- Transductive: label unlabelled data
- Non-convex optimisation



09 Imbalanced Data & Metrics

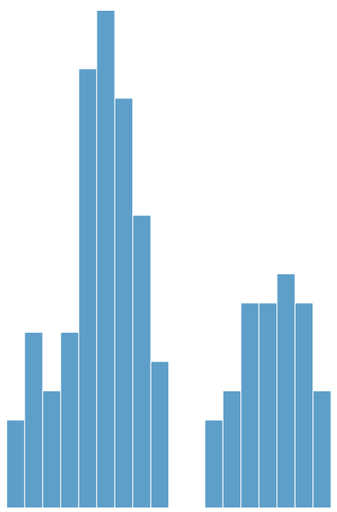
Strategies and metrics for learning under class imbalance

github.com/ranjiGT/ai-machine-learning-codes

27 Class Imbalance Problem

$Acc_{majority\text{-}only} \approx \frac{N_{maj}}{N}$

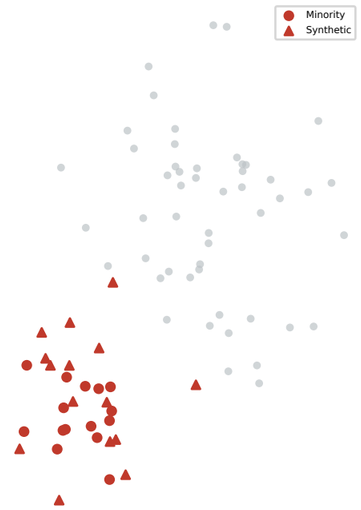
- Minority class often more important
- Accuracy misleads on skewed data
- Solutions: resampling, cost-sensitive learning



28 SMOTE

$x_{new} = x_i + \lambda(x_{nn} - x_i),\ \lambda \sim U[0, 1]$

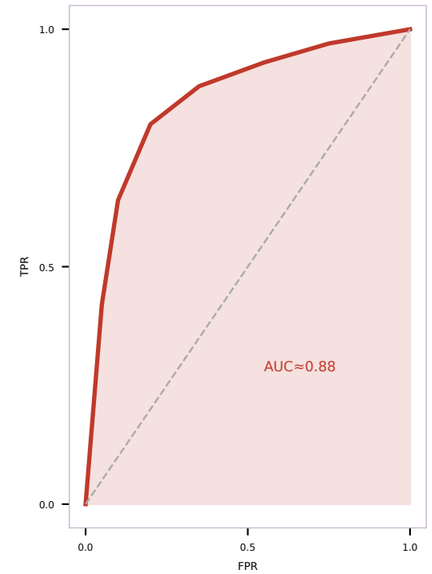
- Synthetic Minority Over-sampling Technique
- Interpolate between minority neighbours
- Avoids exact duplicates



29 ROC and AUC Curves

$AUC = \int_0^1 TPR\, d(FPR)$

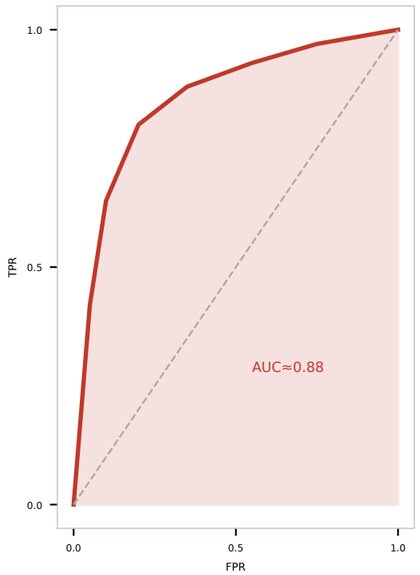
- TPR vs FPR at each threshold
- AUC = P(score_pos > score_neg)
- AUC=1 perfect, 0.5 random



30 ISO Accuracy Lines

$TPR = (1 + \frac{1-\pi}{\pi})FPR - \frac{(1-\pi)Acc}{\pi}$

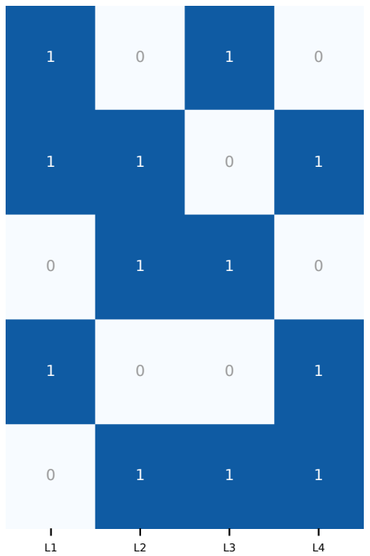
- Contours of equal accuracy on ROC
- Slope = (1−π)/π (prior ratio)
- Visualise cost-sensitive trade-offs



63 Multi-label Eval. Metrics

$HL = \frac{1}{n|L|}\sum_{i,j} y_{ij} \oplus \hat{y}_{ij}$

- Hamming loss: avg label error
- Subset accuracy: exact match ratio
- F1-micro/macro for label imbalance



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

10 Clustering

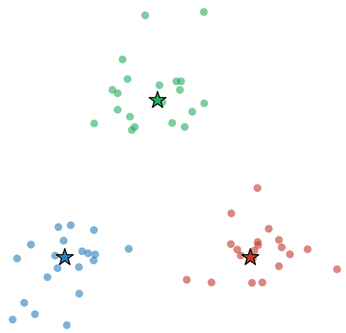
Unsupervised discovery of natural groupings in unlabelled data

github.com/ranjiGT/ai-machine-learning-codes

31 Unsupervised Clustering

$J = \sum_k \sum_{x \in C_k} \|\mathbf{x} - \mu_k\|^2$

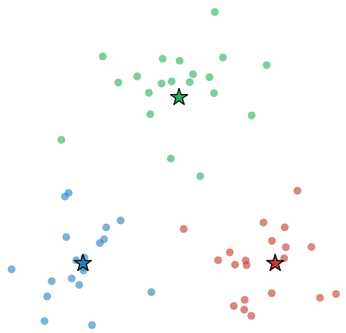
- No labels — find intrinsic groups
- Compactness (within) vs separation (between)
- Many algorithms: K-means, DBSCAN, etc.



32 K-Means Clustering

$\mu_k = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} \mathbf{x}$

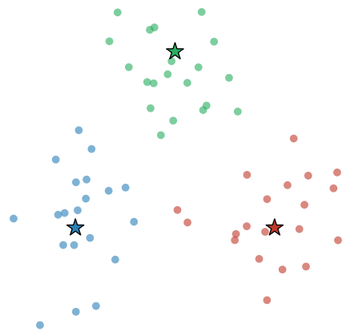
- Assign to nearest centroid; re-compute
- Convergence guaranteed, not global opt
- k chosen via elbow / silhouette



33 Bisecting K-Means

Bisect: pick k with max SSE

- Start with one cluster; bisect worst
- Produces hierarchical structure
- SSE guides split choice



34 Cluster Similarity

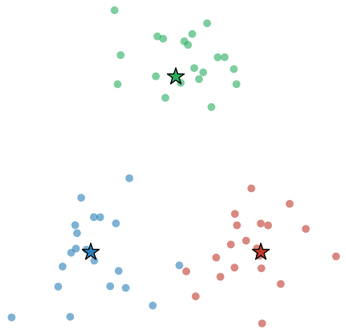
$\text{avg} = \frac{1}{|A||B|} \sum_{a \in A, b \in B} d(a, b)$

- Single, complete, average, Ward linkage
- Proximity matrix between pairs
- Cosine sim for high-dim text

35 Fuzzy C-Means

$u_{ik} = \left(\sum_j \left(\frac{d_{ik}}{d_{jk}} \right)^{2/(m-1)} \right)^{-1}$

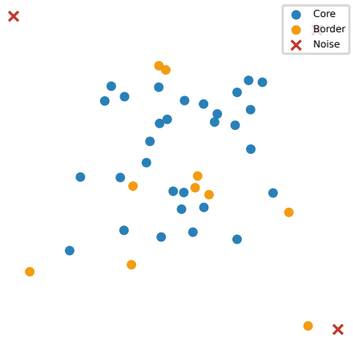
- Each point has membership degree ∈[0,1]
- Minimise weighted SSE
- Reduces to K-means for m→1



36 DBSCAN

$N_\varepsilon(p) = \{q : d(p, q) \le \varepsilon\}$

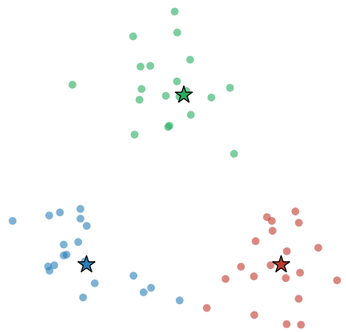
- Core: ≥MinPts in ε-neighbourhood
- Finds arbitrary-shape clusters
- Marks outliers as noise



37 Mean-Shift Clustering

$m(x) = \frac{\sum_{x_i \in N(x)} K(x_i - x) x_i}{\sum_{x_i \in N(x)} K(x_i - x)}$

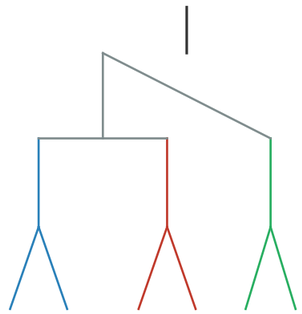
- Shift each point to mean of neighbours
- No k needed; finds modes of density
- Bandwidth h controls granularity



38 Hierarchical Clustering

$d(A \cup B, C) = \frac{|A| d(A, C) + |B| d(B, C)}{|A| + |B|}$

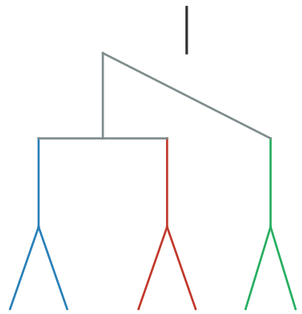
- Agglomerative: bottom-up merging
- Divisive: top-down splitting
- Visualised as dendrogram



39 Linkage Measures

$d_{ward} = \sqrt{\frac{2 n_A n_B}{n_A + n_B}} \|\mu_A - \mu_B\|$

- Single: nearest pair (chaining)
- Complete: farthest pair (compact)
- Ward: min variance increase



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

11

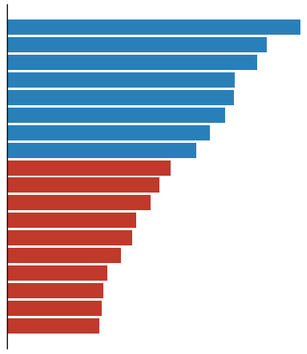
Cluster Validation

Quantitative criteria for assessing cluster quality

40 Cluster Validation

$Q = f(\text{cohesion}, \text{separation})$

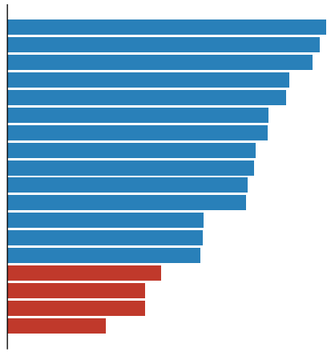
- Internal: cohesion + separation (no labels)
- External: compare to ground truth
- Relative: compare among clusterings



41 Cohesion & Separation

$SEP = \frac{1}{k}\sum_{k} \min_{j \neq k} d(\mu_k, \mu_j)$

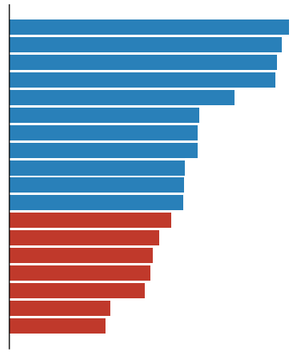
- Cohesion: avg within-cluster dist
- Separation: avg between-cluster dist
- High cohesion + high separation = good



42 Silhouette Coefficient

$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$

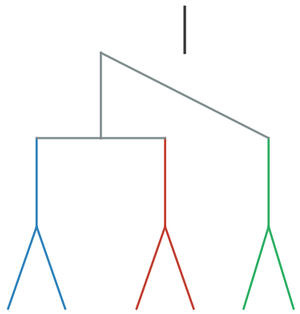
- s(i)=(b(i)−a(i))/max(a,b)
- a: avg within-cluster dist
- b: avg nearest-cluster dist



43 Cophenetic Correlation

$r = \frac{\sum(z - \bar{z})(d - \bar{d})}{\sqrt{\sum(z - \bar{z})^2 \sum(d - \bar{d})^2}}$

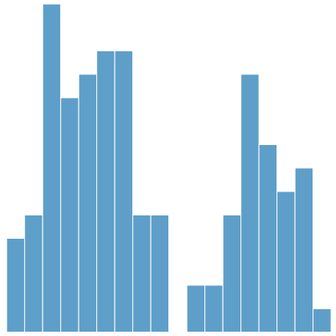
- Correlation: pairwise dists vs tree dists
- Close to 1 = dendro faithful
- Measures hierarchical quality



44 Hopkins Statistic

$H = \frac{\sum u_i^d}{\sum u_i^d + \sum w_i^d}$

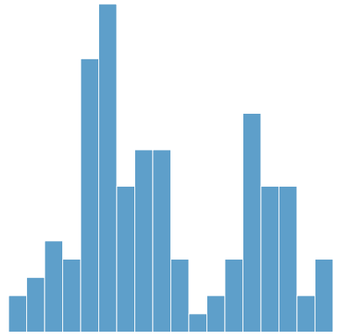
- Tests if data is uniformly distributed
- H close to 0.5 → random
- H close to 1 → clustered structure



45 Cluster Purity

$P = \frac{1}{N}\sum_{k} \max_{j} |C_k \cap T_j|$

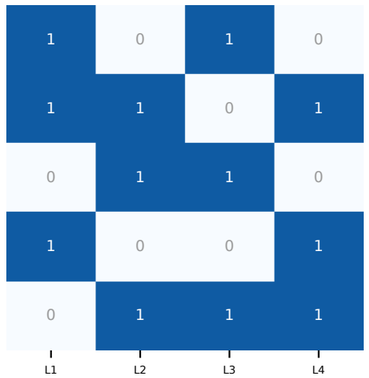
- Assign each cluster to majority class
- Purity = fraction correctly assigned
- Cannot capture over-segmentation



46 Rand & Jaccard Index

$RI = \frac{TP + TN}{TP + FP + FN + TN}$

- Rand: (TP+TN)/all pairs
- Jaccard: TP/(TP+FP+FN)
- Adjusted Rand corrects for chance



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

12

Active Learning

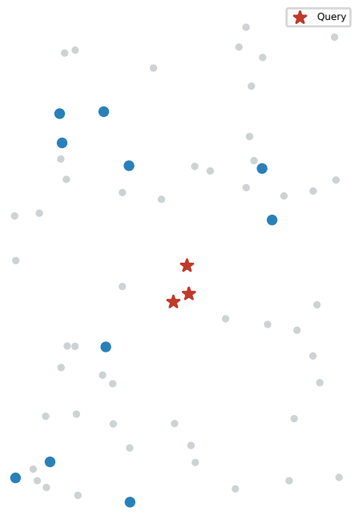
Interactive learning that queries the most informative unlabelled examples

github.com/ranjiGT/ai-machine-learning-codes

47 Active Learning Intro

$x^* = \arg\max_{x \in \mathcal{U}} \phi(x)$

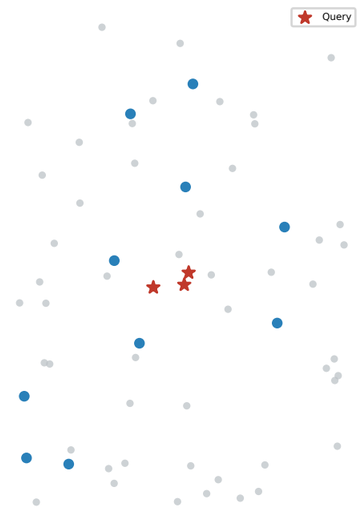
- Label most informative examples first
- Reduces annotation cost significantly
- Query strategies: uncertainty, QBC, EER



48 Membership Query Synthesis

$x^* = \arg\max_{x} \phi(x),\ x$ generated

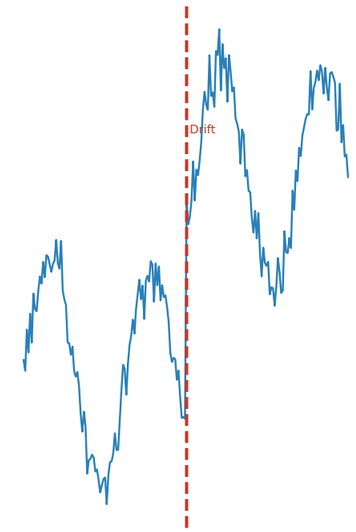
- Learner generates its own queries
- Can query any point in input space
- Risk: unnatural/unrealistic queries



49 Stream-Based Sampling

label if $\phi(x) \geq \tau$

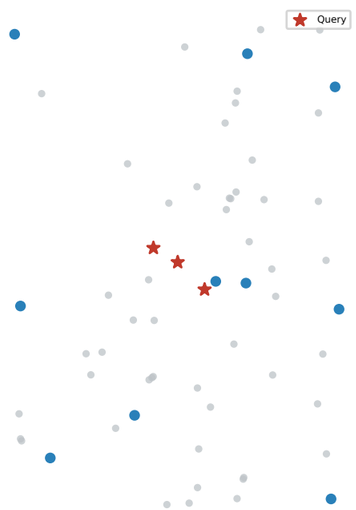
- Each instance arrives once, decide to label
- Accept if informativeness > threshold
- No pool needed; suits streaming data



50 Pool-Based Sampling

$x^* = \arg\max_{x \in \mathcal{U}} \phi(x)$

- Select best query from an unlabelled pool
- Most common paradigm
- Rank pool, pick top-k each round



13

Multi-class & Multi-label

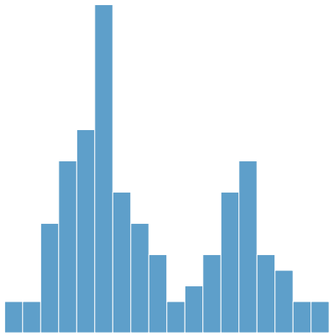
Extending classification beyond binary to multiple classes and labels

github.com/ranjiGT/ai-machine-learning-codes

57 Multi-class: OvR and OvO

$\hat{y} = \arg\max_k f_k(\mathbf{x})$

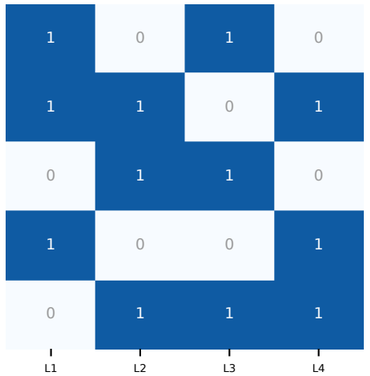
- OvR: one binary clf per class
- OvO: one clf per pair (k(k−1)/2)
- OvR faster; OvO more accurate on some tasks



58 Multi-label Classification

$h : \mathcal{X} \to 2^{\mathcal{L}}$

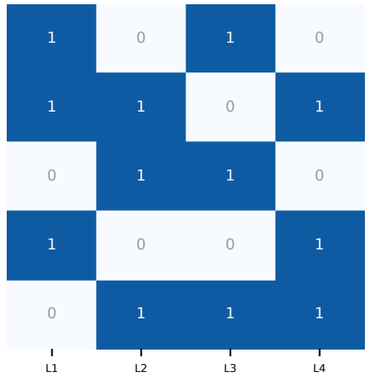
- Each instance can have multiple labels
- Label set y⊆L, |L|>>1
- Hamming loss, F1, subset acc as metrics



59 Problem Transformation

$\mathcal{L} \to \{0,1\}^{|\mathcal{L}|}$

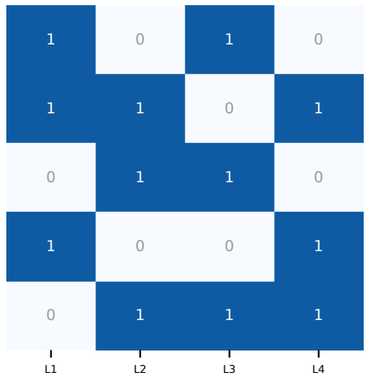
- Convert multi-label to binary problems
- Binary Relevance, Label Powerset, RAkEL
- No modification of base classifier needed



60 Binary Relevance

$H_j(x) = h_j(x),\ j = 1 \ldots |\mathcal{L}|$

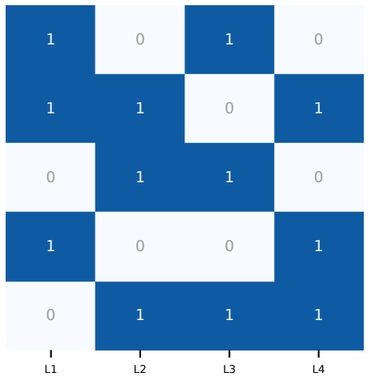
- One binary classifier per label
- Ignores label correlations
- Simple, parallelisable



61 Label Powerset

$H(x) = \text{decode}(h(x))$

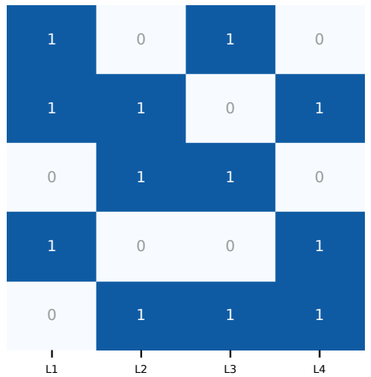
- Each unique label subset becomes a class
- Captures full label correlations
- Exponential classes if many labels



62 RAkEL

$H(x) = \text{vote}\{LP_{S_m}(x)\}_{m=1}^{M}$

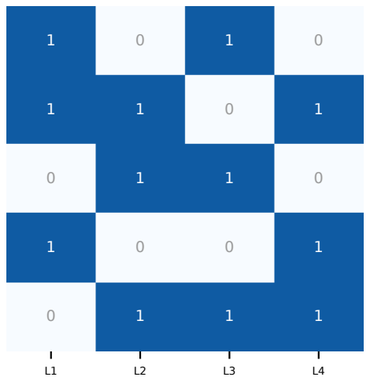
- Random k-label subsets + Label Powerset
- Ensemble of LP classifiers
- Balances correlation and scalability



64 Multi-Target Classification

$h : \mathcal{X} \to \mathcal{Y}_1 \times \cdots \times \mathcal{Y}_T$

- Multiple correlated discrete outputs
- Exploit inter-output correlations
- Chain classifiers, joint output models



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

14

Association Rules

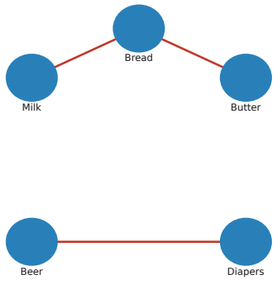
Mining frequent patterns and co-occurrence rules in transactional data

github.com/ranjiGT/ai-machine-learning-codes

66 Association Rules

$\text{conf}(A \to B) = \frac{P(A \cup B)}{P(A)}$

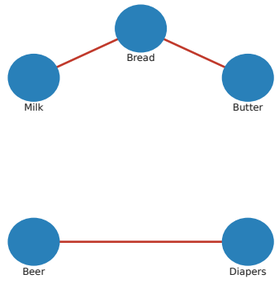
- Rule: {A}→{B}
- Support: P(A∪B)
- Confidence: P(B|A)



67 Apriori Principle

$\sup(X) \geq \text{minsup} \Rightarrow \sup(Y) \geq \text{minsup}\ \forall Y \subseteq X$

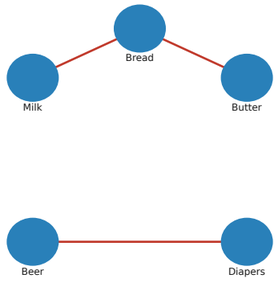
- All subsets of a frequent itemset are frequent
- Prune search space bottom-up
- Generates candidates level by level



68 Item-set Lattice

$\mathcal{F} = \{X : \sup(X) \geq \sigma\}$

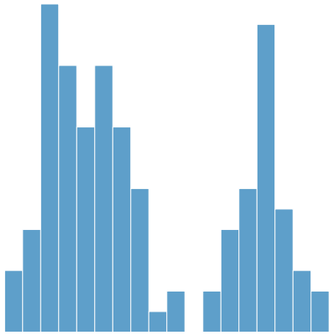
- Lattice of all 2^|I| item subsets
- Frequent itemsets form an ideal
- Monotonicity enables pruning



69 Lift Measure

$\text{lift} = \frac{P(A \cup B)}{P(A)P(B)}$

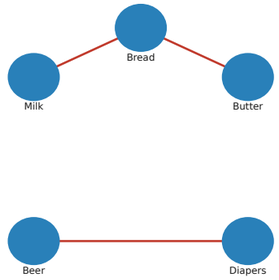
- Lift = conf/(P(B)) — independence baseline
- Lift > 1: positive correlation
- Lift < 1: negative correlation



70 ECLAT

$\sup(A \cup B) = |t(A) \cap t(B)|$

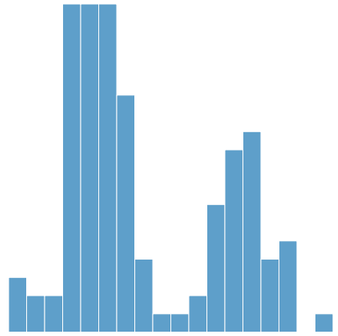
- Vertical representation: item→tid-list
- Intersect tid-lists for support
- Depth-first; efficient for dense data



71 FWER

$\text{FWER} = P\left(\bigcup_i H_{0i} \text{ rejected falsely}\right)$

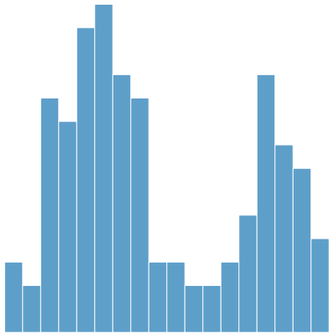
- Family-Wise Error Rate: P(≥1 false positive)
- Bonferroni: α/m per test
- Holm-Bonferroni: step-down correction



72 False Discovery Rate

$FDR = \mathbb{E}\left[\frac{V}{\max(R, 1)}\right]$

- FDR = E[FP/R]: expected false discoveries
- Benjamini–Hochberg procedure
- Less stringent than FWER; more power



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

15

Dimensionality Reduction & Theory

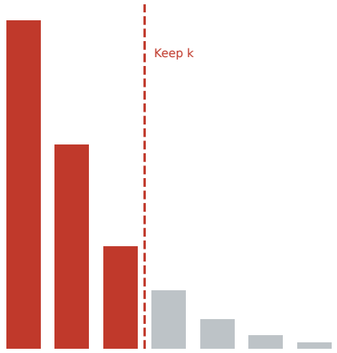
Formal ML theory and methods for compacting high-dimensional representations

github.com/ranjiGT/ai-machine-learning-codes

73 SVD

$\mathbf{M} = \mathbf{U}\Sigma\mathbf{V}^T$

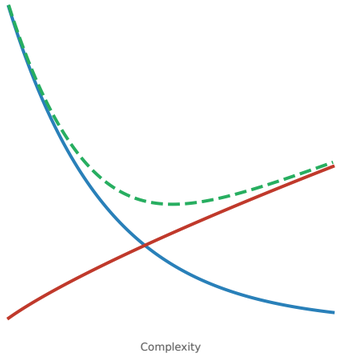
- $M = U\Sigma V^T$ — always exists
- Optimal low-rank approximation (Eckart–Young)
- Basis of LSA, PCA, recommender systems



74 VC Dimension

$R \le \hat{R} + O\left(\sqrt{\frac{d}{n}}\right)$

- Max points a hypothesis class can shatter
- VC(hyperplanes in $\mathbb{R}^n$) = n+1
- PAC bound: E[err] ≤ O(√(d/n))



75 Concept Learning

$h \in VS \Leftrightarrow h$ consistent with D

- Hypothesis: consistent with all examples
- Version space: all consistent hypotheses
- Candidate Elimination refines boundaries

76 Find-S Algorithm

$h \leftarrow h \cup x^+$ (LGG)

- Start most specific; generalise for positives
- Ignores negative examples
- Returns most specific consistent hypothesis

77 Candidate Elimination

$VS = [S, G]$ over the lattice

- Maintains S (specific) and G (general) bounds
- Positive: remove inconsistent G hypotheses
- Negative: remove inconsistent S hypotheses

78 Inductive Bias

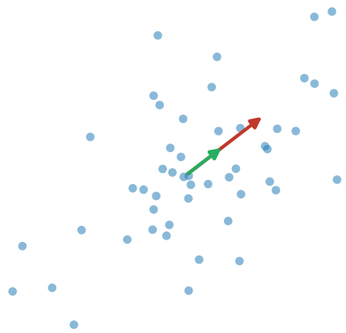
$h \notin VS \Rightarrow$ bias applied

- Assumptions enabling generalisation
- Stronger bias → fewer data needed
- Accuracy depends on bias matching data

86 LDA

$J(\mathbf{w}) = \frac{\mathbf{w}^T\mathbf{S}_B\mathbf{w}}{\mathbf{w}^T\mathbf{S}_W\mathbf{w}}$

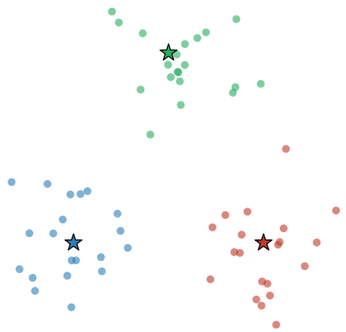
- Maximise between-class / within-class scatter
- Fisher's criterion for projection
- Both classification and dimensionality reduction



87 t-SNE

$KL(P\|Q) = \sum_{i \ne j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$

- Non-linear 2D/3D embedding
- Preserves local neighbour structure
- Perplexity controls neighbourhood size



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

16

Regression Variants

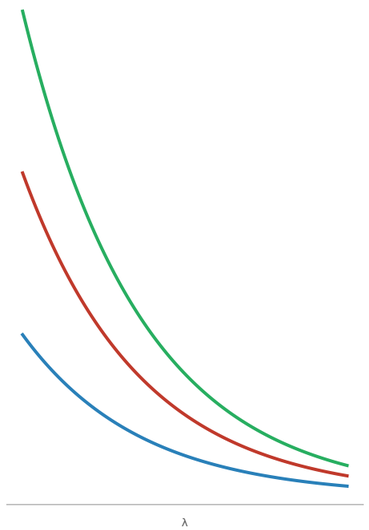
Regularised and non-standard regression methods

github.com/ranjiGT/ai-machine-learning-codes

81 Ridge Regression

$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$

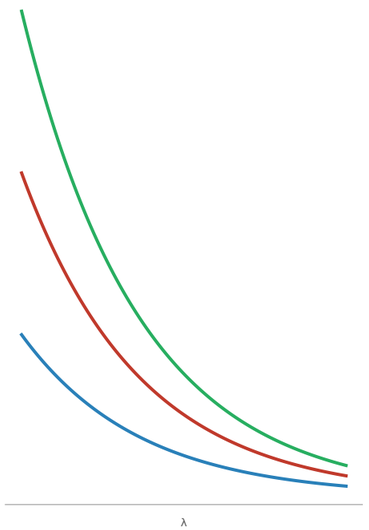
- L2 penalty: $\lambda\Sigma w_i^2$
- Shrinks all coefficients toward 0
- Closed-form solution exists



82 Lasso Regression

$\min\|y - X\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|_1$

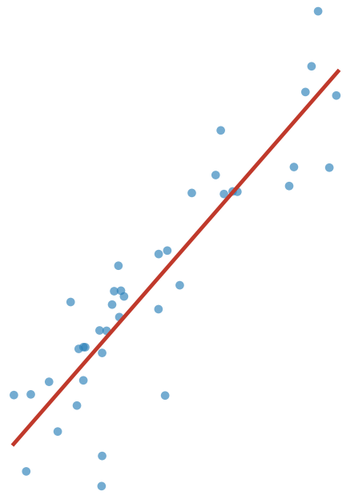
- L1 penalty: $\lambda\Sigma|w_i|$
- Exact zero coefficients → feature selection
- No closed form — use coordinate descent



83 Polynomial Regression

$y = w_0 + w_1x + w_2x^2 + \cdots + w_dx^d$

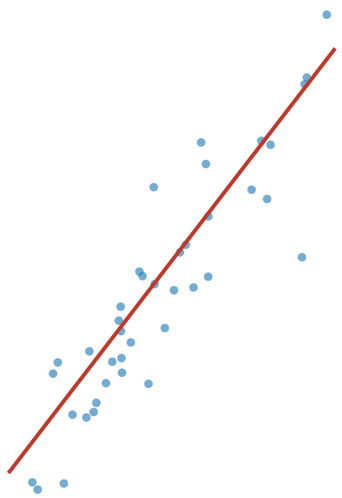
- Add x^2, x^3, … as new features
- Still linear in the parameters
- High degree → overfitting



84 Isotonic Regression

$\min\sum(y_i - \hat{y}_i)^2$ s.t. $\hat{y}_i \le \hat{y}_{i+1}$

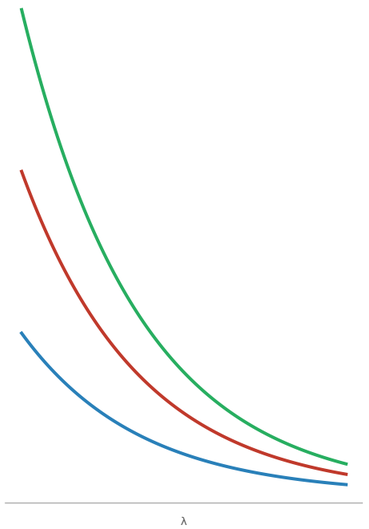
- Fits non-decreasing (monotone) function
- Pool Adjacent Violators algorithm
- Useful for calibration



85 Elastic Net

$\lambda\left[\rho\|\mathbf{w}\|_1 + \frac{1-\rho}{2}\|\mathbf{w}\|_2^2\right]$

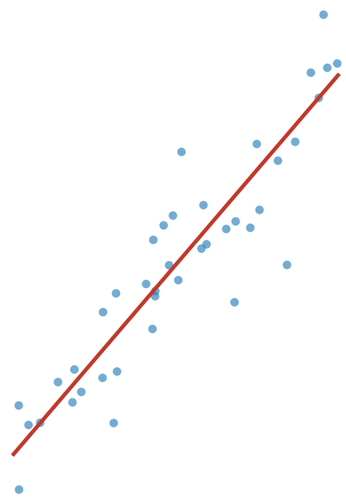
- Combines L1 and L2 penalties
- Groups correlated features together
- Two hyperparameters: α, ρ



109 Linear Learning Machines

$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x} + b)$

- Perceptron, LMS, Widrow–Hoff
- Convergence theorem for separable data
- Foundation of deep learning



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

17

Lazy & Instance-Based Learning

Models that defer computation to query time and memorise training data

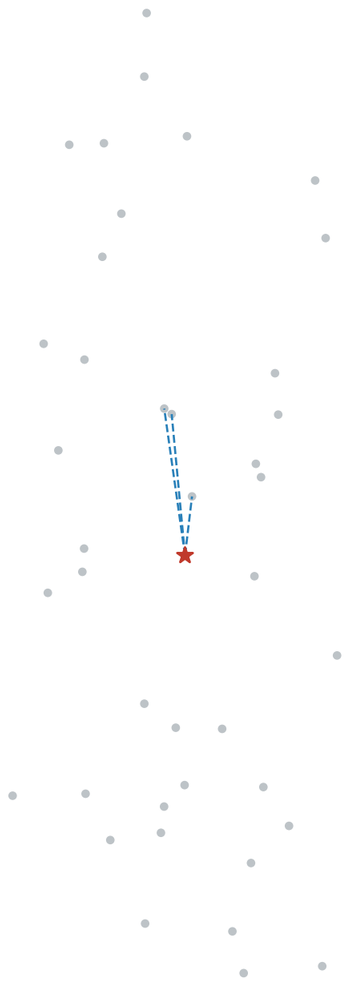
github.com/ranjiGT/ai-machine-learning-codes

88

Lazy Learning

$$\hat{y} = \arg\max_{y} \sum_{x_i \in N(x)} \mathbf{1}[y_i = y]$$

- No explicit training phase
- All computation at query time
- High memory; flexible to local structure

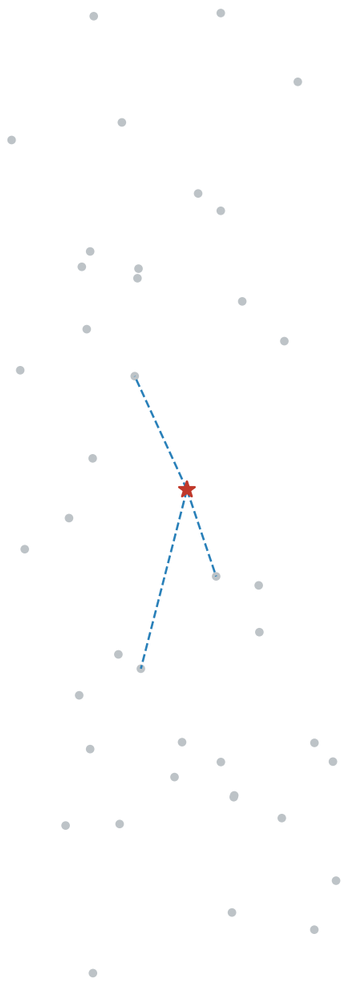


89

Weighted K-NN

$$\hat{y} = \arg\max_{y} \sum_{i \in N(x)} \frac{y_i = y}{d(x, x_i)^2}$$

- Closer neighbours get more weight
- $w_i = 1/d(x,x_i)^2$ common
- Smoother decision boundaries

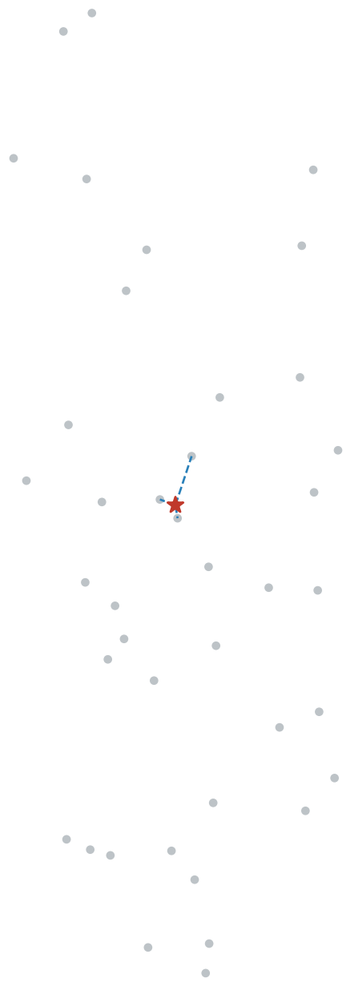


90

Case-Based Reasoning

$$\mathrm{sim}(c_1, c_2) = \sum_i w_i f(c_1^i, c_2^i)$$

- Retrieve similar past cases
- Adapt solution to current problem
- 4R cycle: Retrieve, Reuse, Revise, Retain



Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

18 Stream & Adaptive Learning

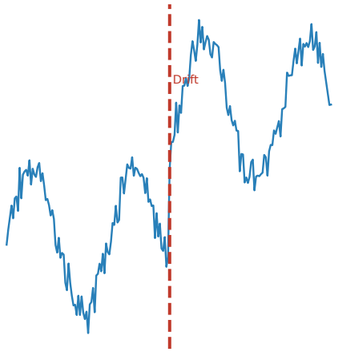
Algorithms that learn from infinite data streams with concept drift

github.com/ranjiGT/ai-machine-learning-codes

117 Stream-Based & Adaptive Learning

$\forall t: |\mathcal{M}_t| \leq M$

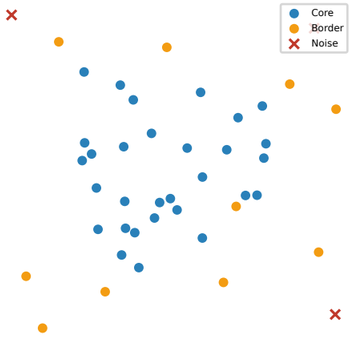
- One-pass over infinite data
- Bounded memory O(1) or O(k)
- Drift detectors trigger model update



118 DenStream

$w = \sum_j 2^{-\lambda(t_c - t_j)}$

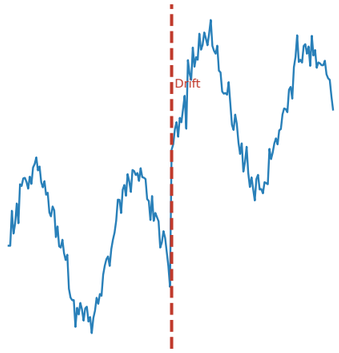
- Fading function f(t)=2^{−λt}
- p-MC / o-MC micro-clusters
- Offline DBSCAN on p-MC centroids



119 ADWIN

$|\hat{\mu}_0 - \hat{\mu}_1| > \varepsilon_{cut}$

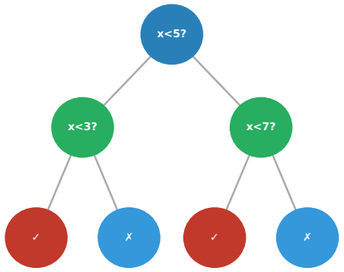
- Adaptive sliding window detector
- Hoeffding-bound based cut test
- Removes old data on drift detection



120 VFDT & CVFDT

$\varepsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}}$

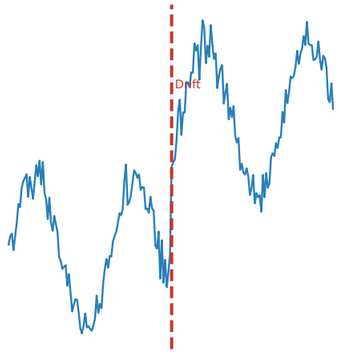
- Hoeffding bound: use n samples to pick split
- CVFDT adds sliding-window adaptation
- Continuous monitoring of best split



121 Concept Drift Detection

$p_t + s_t > p_{min} + k \cdot s_{min}$

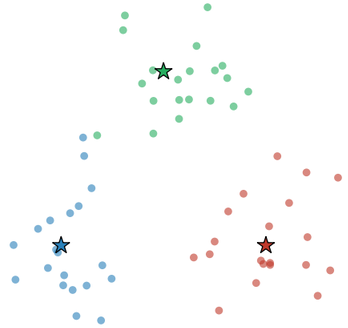
- DDM: error rate + std dev threshold
- EDDM: inter-error distance grows
- Page-Hinkley: cumulative sum test



122 MONIC

$o(C_i, C_j) = \frac{|C_i \cap C_j|}{|C_i|}$

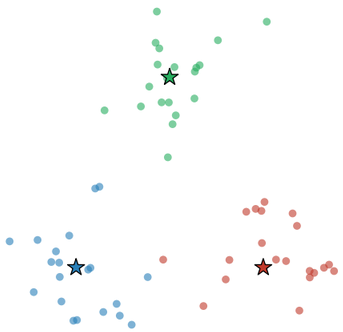
- Tracks cluster transitions over time windows
- Overlap: $|C_i \cap C_j|/|C_i|$
- Survive/split/merge/appear/disappear events



125 CluStream

$\text{RMSD} = \sqrt{\frac{CF2^x}{n} - \left(\frac{CF1^x}{n}\right)^2}$

- CF sufficient stats: CF1x, CF2x, CF1t, CF2t, n
- Online: RMSD boundary assignment
- Offline: weighted k-means on micro-centroids



19

Time Series & Explainability

Classical forecasting and model-agnostic explanation methods

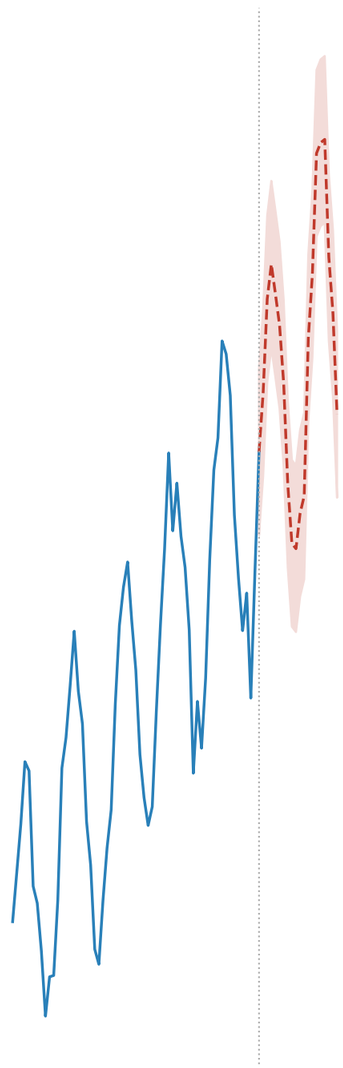
github.com/ranjiGT/ai-machine-learning-codes

116

Advanced Time Series ML

$X_t = f(X_{t-1}, \ldots, X_{t-p})$

- Feature engineering: lag, rolling stats
- Tree-based & NN models for TS
- Cross-validation must respect time order

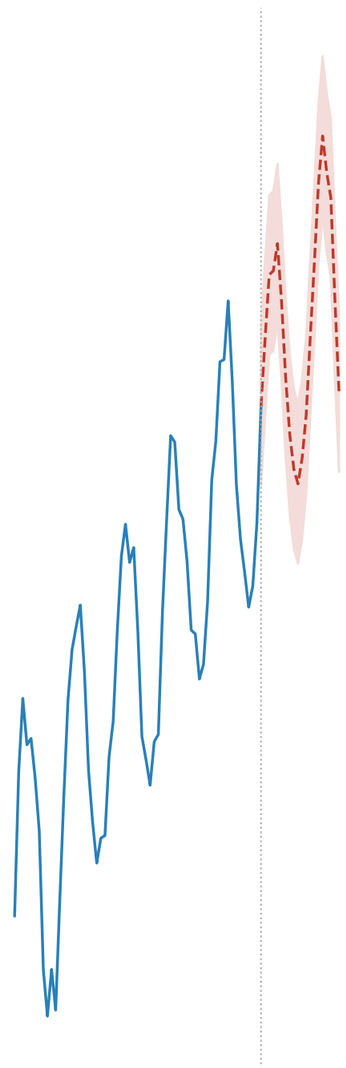


124

ARIMA & SARIMA

$\phi(B)(1-B)^d X_t = \theta(B)\varepsilon_t$

- AR(p): past values; MA(q): past errors
- I(d): differencing for stationarity
- SARIMA adds seasonal (P,D,Q,s) terms

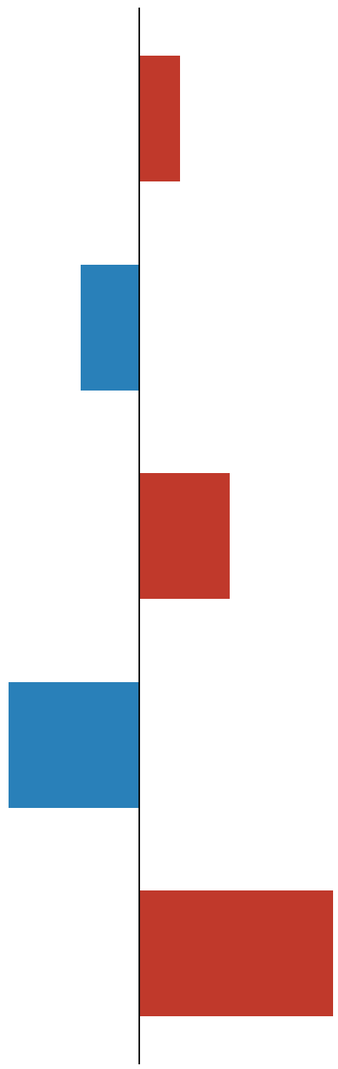


123

SHAP Values

$\phi_i = \sum_{S \subseteq F \setminus i} \frac{|S|!\,(|F|-|S|-1)!}{|F|!}[v(S \cup i) - v(S)]$

- Shapley value: fair credit attribution
- Efficiency: $\Sigma\varphi_i = f(x) - E[f]$
- TreeExplainer: exact $O(TLD^2)$



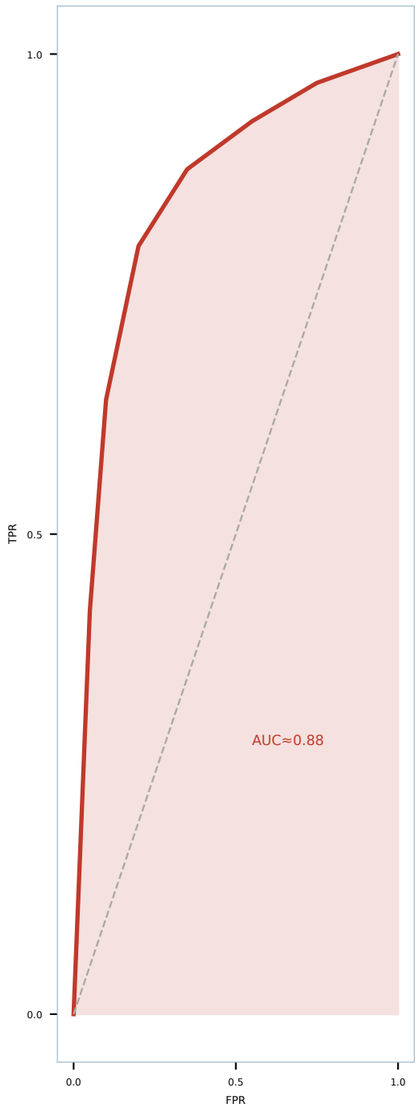
Ranjith Raj Valavanthara · CC BY-NC 4.0 · 2026

Capstone & Projects

End-to-end ML system design, including data, modelling, and deployment

115

End-to-End Real-World ML Project

Pipeline: $\mathcal{D} \to f_\theta \to \hat{y}$

- EDA → feature engineering → modelling
- Baseline → tuning → ensembling
- Evaluation on hold-out; deployment pipeline